

Push_swap

「Swap_push」より自然な名前だから

概要

このプロジェクトでは、限られた命令セットを使って、できるだけ少ないアクション数でスタック上のデータをソートします。

成功するためには、様々な種類のアルゴリズムを操作し、最適なデータソートのために（多くの選択肢の中から）最も適切な解法を選ぶ必要があります。

このプロジェクトはちょうど2人の学習者で完成させるグループ活動です。

バージョン: 1.1

第1章 まえがき

コンピュータサイエンスの神秘の地で、かつてドナルド・クヌース氏がアルゴリズムのスケールについて語るために Big-O 記法を広めました。Big-O とは基本的に、「あなたのコードは永遠に速く動き続けるか.....あるいは、コーヒーを飲んで老いるまで待てるほど遅くなるか」をお上品に表現した方法です。

伝説によれば、初期のプログラマーたちには Big-O がなかったため、コードが遅すぎると単にハードウェアのせいにしていたそうです。そこに Big-O が登場し、CPU のホイールに何匹ハムスターを入れても、アルゴリズム自体が悪ければどうにもならないことを数学的に証明し、楽しみを台無しにしました。

これがなぜ `push_swap` に関係するのか？ あなたは今、2つのスタックと限られた操作しか持たず、眠れない夜の挿入ソートより速く数値をソートしようとしているからです。Big-O は $O(n \log n)$ の優れた戦略が $O(n^2)$ の拙い戦略を入力が大きくなるにつれていかに圧倒するか、その残酷な真実を直視させてくれます。もちろん数学を無視して、操作カウントが爆増するのを眺めることを選ぶなら話は別ですが。

アルゴリズムを設計する際には覚えておいてください：Big-O はあなたを怖がらせるためにあるのではありません——こんなコードを書く人にならないよう防ぐためにあるのです：

```
pb; pa; pb; pa; pb; pa; ...
```

これを1万回繰り返して、なぜチェッカーに嫌われるのか不思議に思う羽目になります。賢くソートしましょう、遅くではなく。

Big-O はポップコーンの SML 表示ではありません..... でももしそうなら、 $O(n \log n)$ が一番ちょうどいいサイズでしょう。

第Ⅱ章 共通事項

- プロジェクトはC言語で書かれていなければなりません。
- プロジェクトはNormに従って書かれていなければなりません。ボーナスファイル/関数がある場合もNormチェックの対象となり、Normエラーがあれば0点になります。
- 関数は未定義動作を除いて、予期せず終了（セグメンテーションフォルト、バスエラー、ダブルフリー等）してはなりません。これが発生した場合、プロジェクトは機能していないとみなされ、評価で0点になります。
- ヒープに割り当てたメモリはすべて適切に解放されなければなりません。メモリリークは容認されません。
- 課題が要求する場合、-Wall、-Wextra、-Werror フラグを使ってccでソースファイルを必要な出力にコンパイルするMakefileを提出しなければなりません。また、Makefileは不要な再リンクを行ってはなりません。
- Makefileには少なくとも\$(NAME)、all、clean、fclean、reのルールが含まれていなければなりません。
- ボーナスを提出するには、Makefileにbonusルールを含める必要があります。ボーナスは課題で特に指定がない限り、_bonus.{c/h}ファイルに配置してください。必須部分とボーナス部分の評価は別々に行われます。
- プロジェクトでlibftの使用が許可されている場合、そのソースと関連するMakefileをlibftフォルダにコピーしてください。プロジェクトのMakefileはそのMakefileを使ってライブラリをコンパイルしてから、プロジェクトをコンパイルしなければなりません。
- プロジェクトのテストプログラムを作成することを推奨します（提出不要・採点対象外）。自分のコードや仲間のコードを簡単にテストする機会になります。防衛（評価）での使用が特に役立ちます。
- 指定されたGitリポジトリに作業を提出してください。Gitリポジトリ内の作業のみが採点されます。Deepthoughtが採点する場合、ピア評価後に行われます。採点中にエラーが発生した場合、評価はその時点で停止します。

第Ⅲ章 AI 使用に関する指示

● 背景

学習の旅において、AI は様々なタスクを支援できます。AI ツールの多様な機能と、それがあなたの作業をどのようにサポートできるかを探ってください。ただし、常に注意を払い、結果を批判的に評価してください。コード、ドキュメント、アイデア、技術的な説明のいずれであれ、質問が適切だったか、生成されたコンテンツが正確かを完全に確認することはできません。仲間はミスや盲点を避ける上で貴重なリソースです。

● 主要メッセージ

- AI を使って反復的・退屈なタスクを減らしましょう。
- コーディングと非コーディング両方のプロンプティングスキルを磨きましょう——これは将来のキャリアに役立ちます。
- AI システムの仕組みを学び、一般的なリスク・偏り・倫理的問題をよりよく予測し回避しましょう。
- 仲間と協力して、技術スキルと対人スキルの両方を継続的に磨きましょう。
- AI 生成コンテンツは、完全に理解し責任を持てるものだけを使用しましょう。

● 学習者のルール

- AI ツールを探り、その仕組みを理解するための時間を取り、倫理的に使用し潜在的な偏りを軽減できるようにしましょう。
- プロンプトを作成する前に問題を深く考えましょう——より明確で詳細、適切な語彙を使った関連性の高いプロンプトを書く助けになります。
- AI 生成物はすべてを体系的に確認・見直し・疑問視・テストする習慣を身につけましょう。
- 常に仲間のレビューを求めましょう——自分だけの検証に頼らないこと。

● フェーズの成果

- 汎用的かつ分野固有のプロンプティングスキルを両方磨く。
- AI ツールを効果的に使用して生産性を向上させる。
- 計算論的思考、問題解決力、適応力、協調性を引き続き強化する。

● コメントと例

- 試験、評価など、本当の理解を示さなければならない場面に定期的に遭遇します。準備を怠らず、技術スキルと対人スキルの両方を磨き続けてください。
- 推論を説明したり仲間と議論したりすることで、理解の不足が明らかになることがよくあります。仲間との学習を優先してください。
- AI ツールはあなたの固有のコンテキストを欠いていることが多く、一般的な回答をしがちです。同じ環境を共有する仲間の方が、より適切で正確な洞察を提供できます。

良い例:

AIに「ソート関数をどうテストすればいい? 」と聞くといくつかのアイデアが出てきます。試してみて仲間と結果を見直します。一緒にアプローチを改善します。

悪い例:

AIにまるごと関数を書かせ、プロジェクトにコピペします。ピア評価の際に何をしているのか、なぜそうするのか説明できず、信頼を失い——プロジェクトに失敗します。

第Ⅳ章 はじめに

Push swap プロジェクトは非常にシンプルでわかりやすいアルゴリズムプロジェクトです：データをソートしなければなりません。

整数値のセット、2つのスタック、そして両方のスタックを操作するための一連の操作が与えられます。

目標は？ 引数として受け取った整数をソートする Push swap の操作で作られた最小のプログラムを計算して標準出力に表示する `push_swap` という C プログラムを書くことです。

簡単？

やってみましょう.....

第Ⅴ章 目的

このプロジェクトの目的は、アルゴリズムの複雑性を非常に具体的な形で発見することです。

数値をソートするのは簡単ですが、2つのスタックと限られた操作だけで速くソートするのは別の話です。完全にランダムなリストのソートとほぼソート済みのリストのソートも、まったく異なる難しさがあります。

アルゴリズムの選択が、素早い勝利と果てしない操作の繰り返しの差を生む様子を、すぐに実感できるでしょう。

第 VI 章 必須部分

VI.1 グループプロジェクトの要件

- このプロジェクトはちょうど 2 人の学習者が協力して完成させなければなりません。
- 両方の学習者がプロジェクトに意味のある貢献をし、実装されたすべてのアルゴリズムを理解していなければなりません。
- リポジトリの README.md ファイルに、両方の学習者の貢献を明確に記載しなければなりません。
- 防衛（評価）の際、両方の学習者が出席し、コードのどの部分も説明できなければなりません。
- プロジェクトの提出物にはリポジトリに両方の学習者のログインが含まれていなければなりません。

VI.2 ルール

- a と b という名前の 2 つのスタックがあります。
- 最初の状態：
 - スタック a には重複のないランダムな量の負および/または正の数値が含まれています。
 - スタック b は空です。
- スタック a 内の数値を昇順にソートすることが目標です。そのために以下の操作を使用できます：

sa (swap a) : スタック a の先頭の 2 要素を入れ替えます。要素が 1 つ以下の場合は何もしません。

sb (swap b) : スタック b の先頭の 2 要素を入れ替えます。要素が 1 つ以下の場合は何もしません。

ss: sa と sb を同時に実行します。

pa (push a) : b の先頭の要素を取って a の先頭に置きます。b が空の場合は何もしません。

pb (push b) : a の先頭の要素を取って b の先頭に置きます。a が空の場合は何もしません。

ra (rotate a) : スタック a の全要素を 1 つ上にシフトします。最初の要素が最後になります。

rb (rotate b) : スタック b の全要素を 1 つ上にシフトします。最初の要素が最後になります。

rr: ra と rb を同時に実行します。

rra (reverse rotate a) : スタック a の全要素を 1 つ下にシフトします。最後の要素が最初になります。

rrb (reverse rotate b) : スタック b の全要素を 1 つ下にシフトします。最後の要素が最初になります。

rrr: rra と rrb を同時に実行します。

VI.3 アルゴリズム要件

アルゴリズムの複雑性（時間・空間）を厳密に理解するために、4つの異なるソート戦略を実装し、push_swap プログラムに統合しなければなりません。プログラムは入力の構成に基づいて実行時に戦略を選択できなければなりません。

VI.3.1 複雑性モデルと操作制約

すべての戦略はCで実装され、ソートを実行するための Push_swap 操作のシーケンスを生成しなければなりません。これは：

- C アルゴリズムは Input を分析し、適切な操作シーケンス (sa、sb、ss、pa、pb、ra、rb、rr、rra、rrb、rrr) を生成します。
- 戦略の出力はスタックをソートするこれらの操作のシーケンスです。
- 複雑性クラスを述べる際は、従来の配列ベースアルゴリズムの理論的複雑性ではなく、生成される Push_swap 操作の数で測定されたコストを反映していなければなりません。

VI.3.2 無秩序度メトリック（必須）

この課題において、無秩序度（disorder）は0から1の間の数値で、初期スタック a がソートされた状態からどれほど離れているかを示します。

数値がすでに正しい順序であれば無秩序度は0です。最悪の順序であれば1です。その間はスタックが部分的にソートされているが、まだ乱れていることを意味します。

計算するには、スタック内のすべての可能なペアを見ます。大きい数値が小さい数値の前に現れるたびに、そのペアは「間違い」としてカウントされます。間違いが多いほど、無秩序度は1に近くなります。

```
function compute_disorder(stack a):
    mistakes = 0
    total_pairs = 0
    for i from 0 to size(a)-1:
        for j from i+1 to size(a)-1:
            total_pairs += 1
            if a[i] > a[j]:
                mistakes += 1
    return mistakes / total_pairs
```

無秩序度は操作を行う前に測定しなければなりません。

VI.3.3 必要な戦略

1. シンプルアルゴリズム ($O(n^2)$) : $O(n^2)$ クラスのベースラインアルゴリズムを少なくとも1つ実装してください。例：
 - 挿入ソートの応用
 - 選択ソートの応用
 - バブルソートの応用
 - シンプルな最小/最大抽出法
2. 中程度アルゴリズム ($O(n\sqrt{n})$) : $O(n\sqrt{n})$ クラスのアルゴリズムを少なくとも1つ実装してください。例：
 - チャンクベースのソート ($\sqrt{n}$ 個のチャンクに分割)
 - ブロックベースのパーティショニング手法

- $\sqrt{n}$ バケットのバケットソート応用
 - 範囲ベースのソート戦略
3. 複合アルゴリズム ($O(n \log n)$) : $O(n \log n)$ クラスのアルゴリズムを少なくとも 1 つ実装してください。例:
- 基数ソートの応用 (LSD または MSD)
 - 2 スタックを使ったマージソートの応用
 - スタックパーティショニングを使ったクイックソートの応用
 - ヒープソートの応用
 - バイナリインデックスツリーのアプローチ
4. カスタム適応型アルゴリズム (学習者の設計) : 測定された無秩序度に応じて異なる内部手法を選択する適応型戦略を設計してください。特定の名前付きアルゴリズムに限定されず、内部技術は完全に自由です。ただし、設計は Push_swap 操作モデルにおいて以下の複雑性目標を満たさなければなりません:

低無秩序度: $\text{disorder} < 0.2$ の場合、選択した手法は $O(n^2)$ で動作しなければなりません。

中程度無秩序度: $0.2 \leq \text{disorder} < 0.5$ の場合、選択した手法は $O(n\sqrt{n})$ で動作しなければなりません。

高無秩序度: $\text{disorder} \geq 0.5$ の場合、選択した手法は $O(n \log n)$ で動作しなければなりません。

リポジトリ (README.md 等) に、閾値の根拠、各レジームで使用する内部技術、Push_swap モデルにおける時間と空間の複雑性引数 (上限) を文書化しなければなりません。

VI.4 例

これらの操作の効果を説明するために、ランダムな整数のリストをソートしましょう。この例では、両方のスタックが右から成長するとします。

```
Init a and b:  2 1 3 6 5 8 | (bは空)
Exec sa:       1 2 3 6 5 8 ← 先頭2つを入れ替え
Exec pb pb pb: a=[6 5 8]  b=[3 2 1]
Exec ra rb (=rr): a=[5 8 6] b=[2 1 3]
Exec rra rrb (=rrr): a=[6 5 8] b=[3 2 1]
Exec sa:       a=[5 6 8]  b=[3 2 1]
Exec pa pa pa: a=[1 2 3 5 6 8] ← ソート完了
```

スタック a の整数が 12 操作でソートされました。もっと少なくできますか？

VI.5 「push_swap」プログラム

プログラム名	push_swap
提出ファイル	Makefile, *.h, *.c
Makefile	NAME, all, clean, fclean, re
引数	スタック a: 整数のリスト
外部関数	read, write, malloc, free, exit、ft_printf または自作同等品
Libft 許可	はい
説明	スタックをソートする

プロジェクトは以下のルールに準拠しなければなりません:

- ソースファイルをコンパイルする Makefile を提出しなければなりません。再リンクしてはなりません。
- グローバル変数は禁止されています。
- push_swap というプログラムを書き、以下の引数を取らなければなりません:
 - スタック a を整数のリストとして (最初の引数がスタックの先頭)。
 - オプションの戦略セクター:
 - simple $O(n^2)$ アルゴリズムの使用を強制
 - medium $O(n\sqrt{n})$ アルゴリズムの使用を強制
 - complex $O(n \log n)$ アルゴリズムの使用を強制
 - adaptive 無秩序度に基づく適応型アルゴリズムの使用を強制 (セクター未指定時のデフォルト)
- プログラムはスタック a をソートするための最小の Push_swap 操作リストを表示しなければなりません (最小値が先頭)。
- 操作は $\ln$ で区切られ、それ以外の文字は含めません。
- 各アルゴリズムで主張する複雑性クラスはこのモデルで有効でなければなりません。
- 戦略の選択はすべての有効な入力で機能しなければなりません。
- パラメーターが指定されない場合、プログラムは何も表示せずプロンプトを返さなければなりません。
- エラーの場合、標準エラーに「Error」に続けて $\ln$ を表示しなければなりません。エラーには整数でない引数、有効範囲外の整数、重複値などが含まれます。
- バイナリには 4 つの戦略 (Simple $O(n^2)$ 、Medium $O(n\sqrt{n})$ 、Complex $O(n \log n)$ 、Adaptive) がすべて組み込まれていなければなりません。
- オプションのベンチマークモード (--bench) はソート後に以下を表示しなければなりません:
 - 計算された無秩序度 (小数点 2 桁の%)
 - 使用された戦略名とその理論的複雑性クラス
 - 総操作数
 - 各操作タイプの数 (sa、sb、ss、pa、pb、ra、rb、rr、rra、rrb、rrr)

ベンチマーク出力は stderr に送られ、フラグが存在する場合のみ表示されます。

VI.5.1 使用例

```
$>./push_swap 2 1 3 6 5 8
ra
pb
rra
pb
... (以下操作が続く)
```

```
$> ARG="4 67 3 87 23"; ./push_swap --adaptive $ARG | wc -l
13
```

```
$> ./push_swap --simple 5 4 3 2 1
rra
pb
... (以下操作が続く)
```

```
$> ARG="4 67 3 87 23"; ./push_swap --complex $ARG | ./checker_linux
$ARG
OK
```

```
$> ./push_swap --adaptive 0 one 2 3
Error
$> ./push_swap --simple 3 2 3
Error
```

VI.6 パフォーマンスベンチマーク

このプロジェクトを検証するため、最小操作数で一定のパフォーマンス目標を達成しなければなりません：

- 100 個のランダムな数値の場合：
 - 合格（最低要件）：2000 操作未満
 - 良好なパフォーマンス：1500 操作未満
 - 優秀なパフォーマンス：700 操作未満
- 500 個のランダムな数値の場合：
 - 合格（最低要件）：12000 操作未満
 - 良好なパフォーマンス：8000 操作未満
 - 優秀なパフォーマンス：5500 操作未満

これらはすべて評価時に提供されたチェッカーを使って検証されます。

第 VII 章 README 要件

README.md ファイルを Git リポジトリのルートに提供しなければなりません。その目的は、プロジェクトに詳しくない人（仲間、スタッフ、採用担当者等）がプロジェクトの内容、実行方法、詳細情報の場所をすぐに理解できるようにすることです。

README.md には少なくとも以下を含めなければなりません：

- 最初の行はイタリック体で「このプロジェクトは 42 カリキュラムの一環として<ログイン 1>[, <ログイン 2>[...]]によって作成されました。」と記載する。
- プロジェクトの目標と概要を明確に示す「説明」セクション。
- コンパイル、インストール、実行に関する関連情報を含む「手順」セクション。
- トピックに関連する参考文献（ドキュメント、記事、チュートリアル等）と、AI の使用方法（どのタスクとどの部分に使用したか）を記載した「リソース」セクション。
- このプロジェクトで選択したアルゴリズムの詳細な説明と正当化も含めなければなりません。

英語が推奨されますが、キャンパスの主要言語を使用しても構いません。

第 VIII 章 ボーナス部分

このプロジェクトはシンプルさゆえに追加機能の機会は限られています。しかし、自分専用のチェッカーを作ってみませんか？

チェッカープログラムを使うと、push_swap プログラムが生成した操作リストが実際にスタックを正しくソートするかどうかを確認できます。

⚠ ボーナス部分は必須部分が完璧な場合のみ評価されます。完璧とは必須部分が完全に完成し、エラーなく機能することを意味します。このプロジェクトでは、例外なくすべてのベンチマークを通過することが必要です。必須要件をすべて満たしていない場合、ボーナス部分は評価されません。

VIII.1 「checker」プログラム

プログラム名	checker
提出ファイル	*.h, *.c
Makefile	bonus
引数	スタック a: 整数のリスト
説明	ソート操作を実行する

- checker というプログラムを書き、スタック a を整数のリストとして引数に取ります。最初の引数がスタックの先頭にあるべきです。引数が与えられない場合、停止して何も表示しません。
- その後、標準入力から操作を読み取り、各命令は\n で終わります。すべての命令が読み取られたら、引数として受け取ったスタックでそれらを実行します。
- 実行後、スタック a が実際にソートされ、スタック b が空であれば、標準出力に「OK」に続けて\n を表示します。
- それ以外の場合は、標準出力に「KO」に続けて\n を表示します。
- エラーの場合は、標準エラーに「Error」に続けて\n を表示します。

```
$>./checker 3 2 1 0
rra
pb
sa
rra
pa
OK
```

```
$>./checker 3 2 1 0
sa
rra
pb
KO
```

```
$>./checker 3 2 one 0
Error
```

⚠ 提供されたバイナリとまったく同じ動作を再現する必要はありません。エラー管理は必須ですが、引数の解析方法はあなたが決めます。

第 IX 章 提出とピア評価

いつものように Git リポジトリに課題を提出してください。リポジトリ内の作業のみが防衛（評価）で採点されます。ファイル名が正しいか再確認することを忘れずに。

グループプロジェクトの提出要件：

- 両方の学習者がリポジトリのコントリビューターとして登録されていなければなりません。
- README.md に両方の学習者の貢献が明確に記載されていなければなりません。
- ピアレビュー防衛の際、両方の学習者が出席しなければなりません。
- 各学習者がコードのどの部分も説明・防衛できなければなりません。

評価中、プロジェクトへの簡単な修正が求められることがあります。これには、わずかな動作変更、数行のコードの記述・書き直し、または簡単に追加できる機能などが含まれます。

この手順はすべてのプロジェクトに適用されるわけではありませんが、評価ガイドラインに記載されている場合は準備しておかなければなりません。

このステップはプロジェクトの特定部分について実際の理解を検証するためのものです。修正はあなたが選んだ任意の開発環境で行うことができ、特定の時間枠が評価の一部として定義されていない限り、数分以内に実施可能なはずです。